

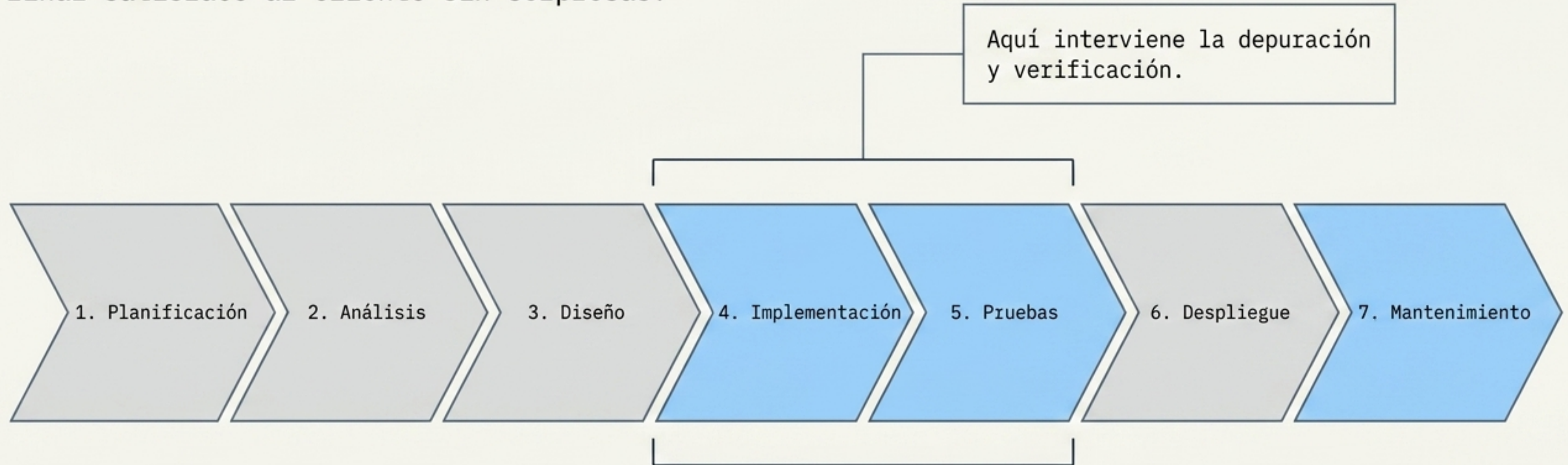
Depuración y Pruebas de Programa

Estrategias, herramientas y metodologías para garantizar la fiabilidad del código.

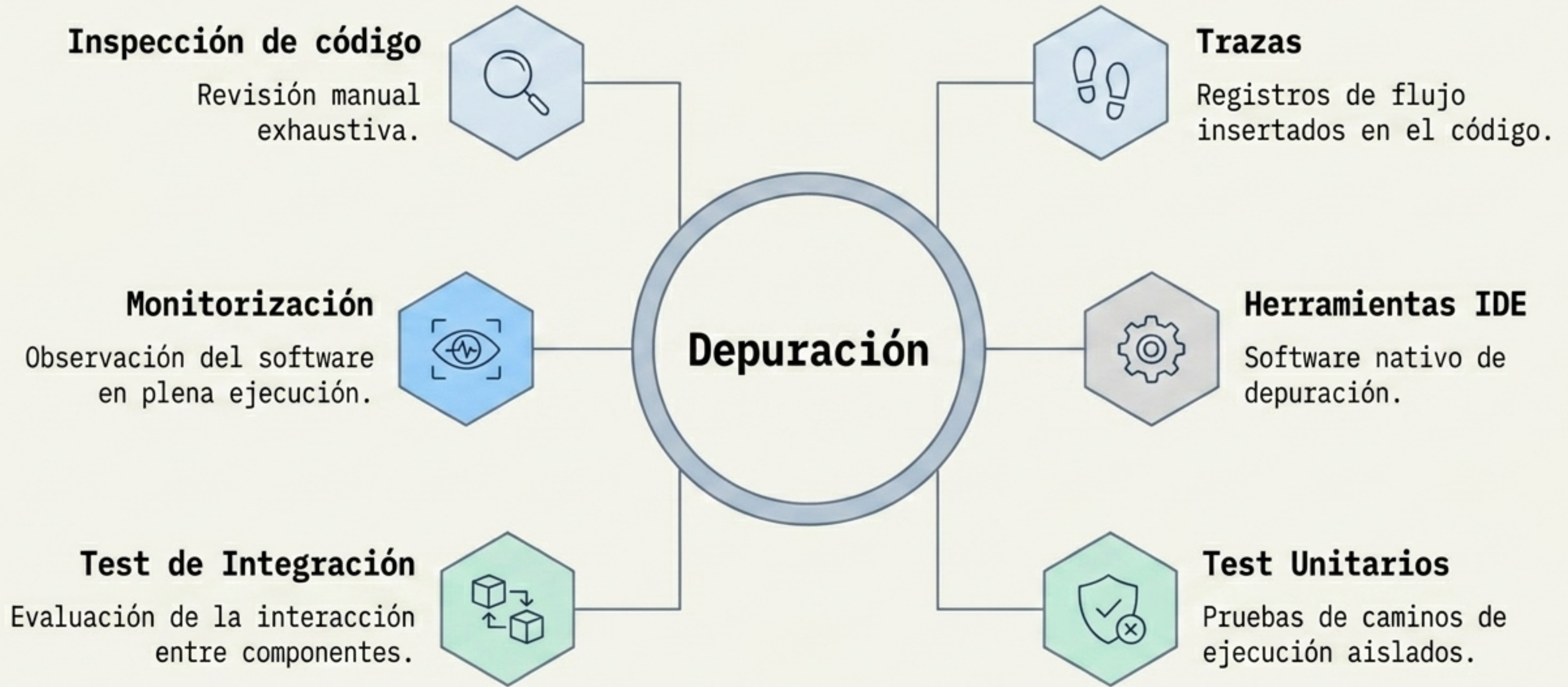


El ciclo de vida del software

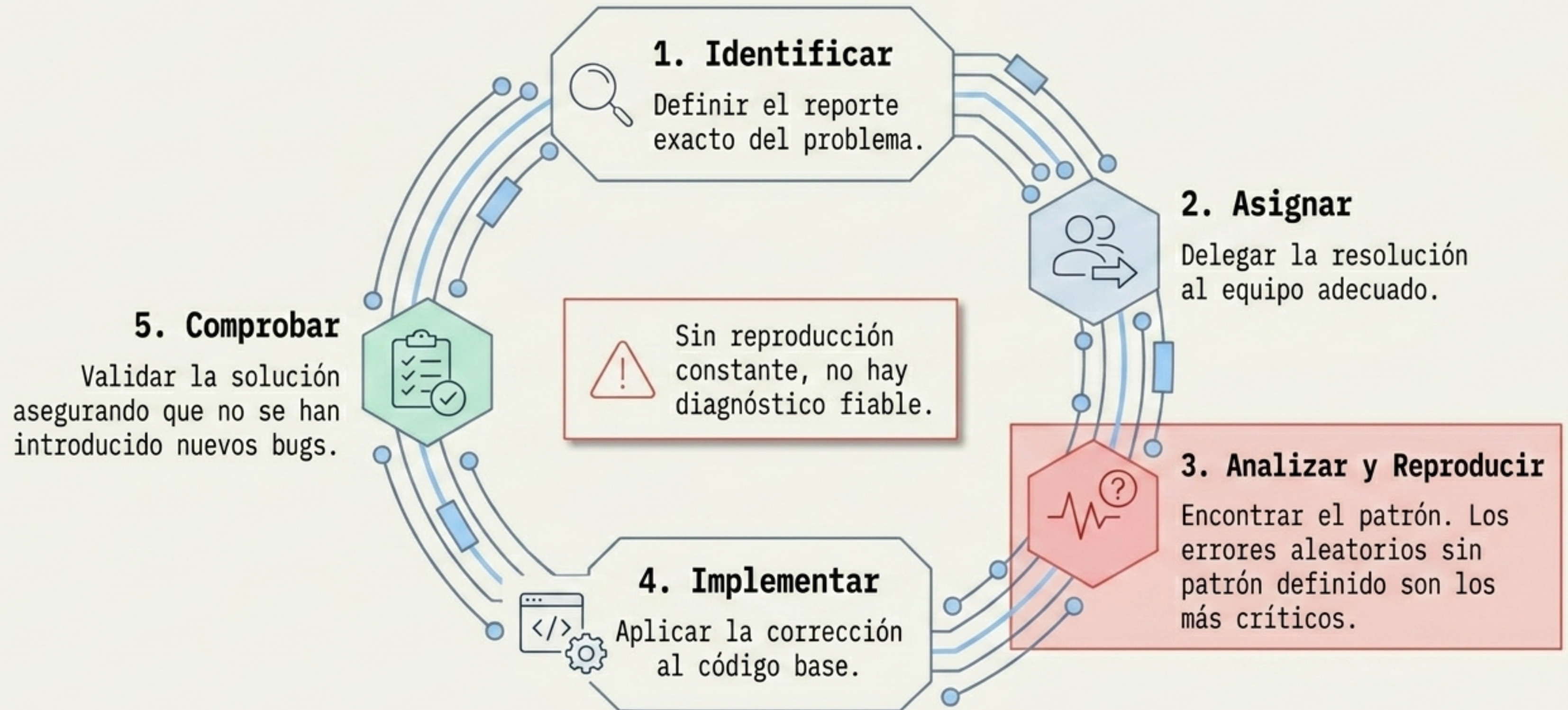
Un modelo sistemático estructurado. Si se ejecuta correctamente en todas sus fases, el producto final satisface al cliente sin sorpresas.



Diagnóstico con lupa: Métodos de depuración



El circuito de la resolución de errores



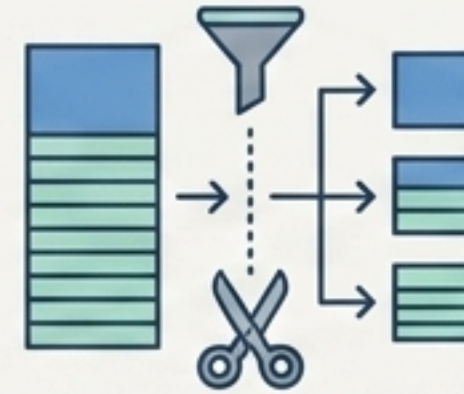
Elección táctica: Estrategias de depuración

Fuerza Bruta



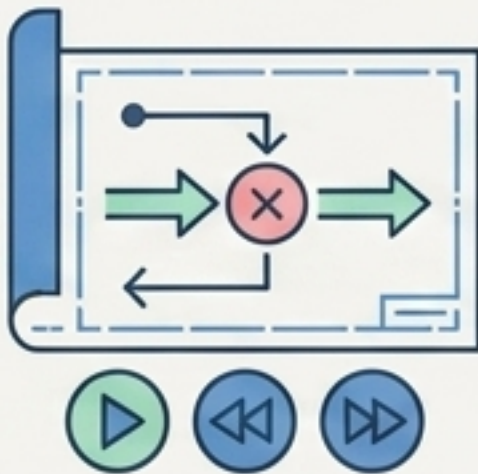
Alto esfuerzo. Exige comprensión total del sistema.
Uso ideal: Causas completamente desconocidas.

Eliminación



Partición de datos. Consiste en aislar el error minimizando gradualmente las entradas y variables.

Análisis de Flujo (Delante/Atrás)



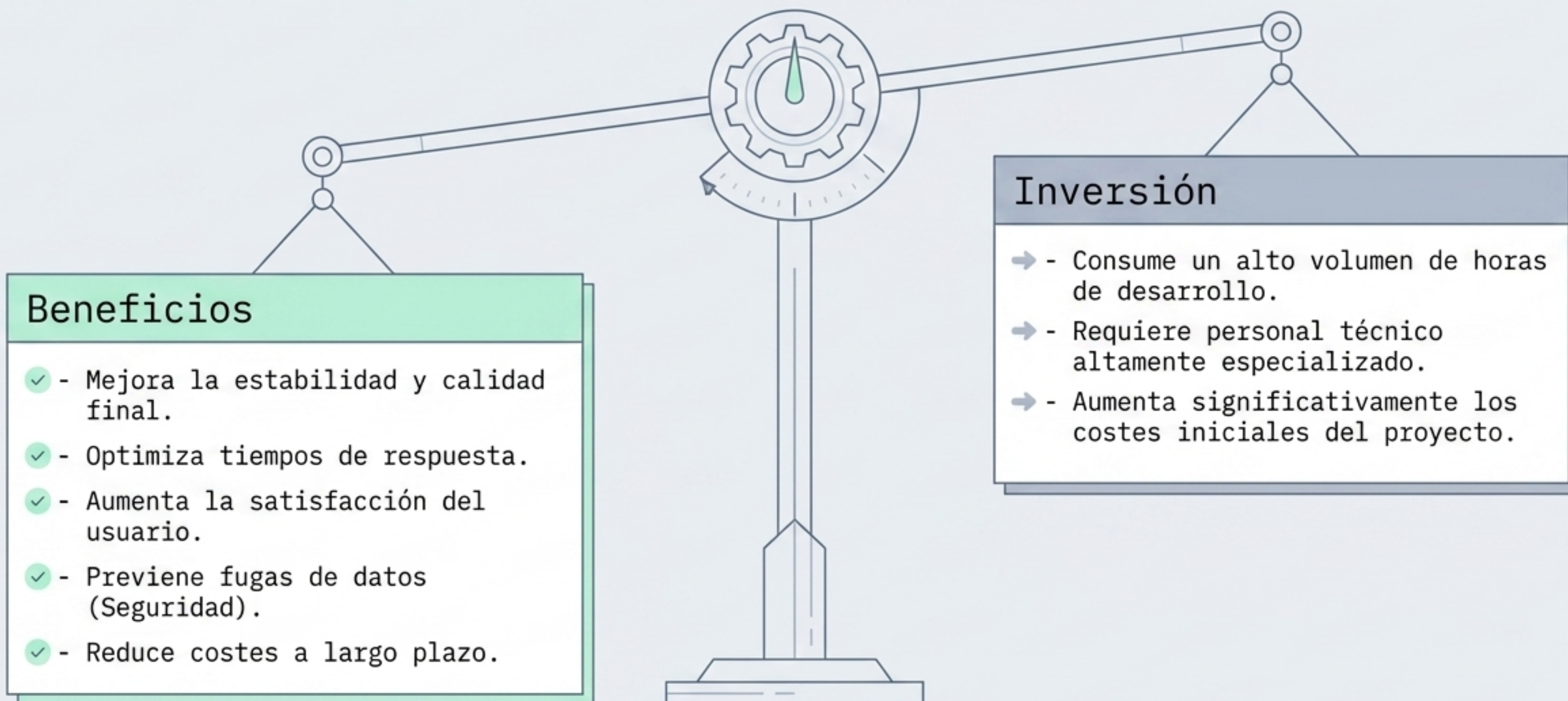
Rastreo temporal. Uso de trazas y breakpoints para pausar, seguir o retroceder en la ejecución del código.

Automática

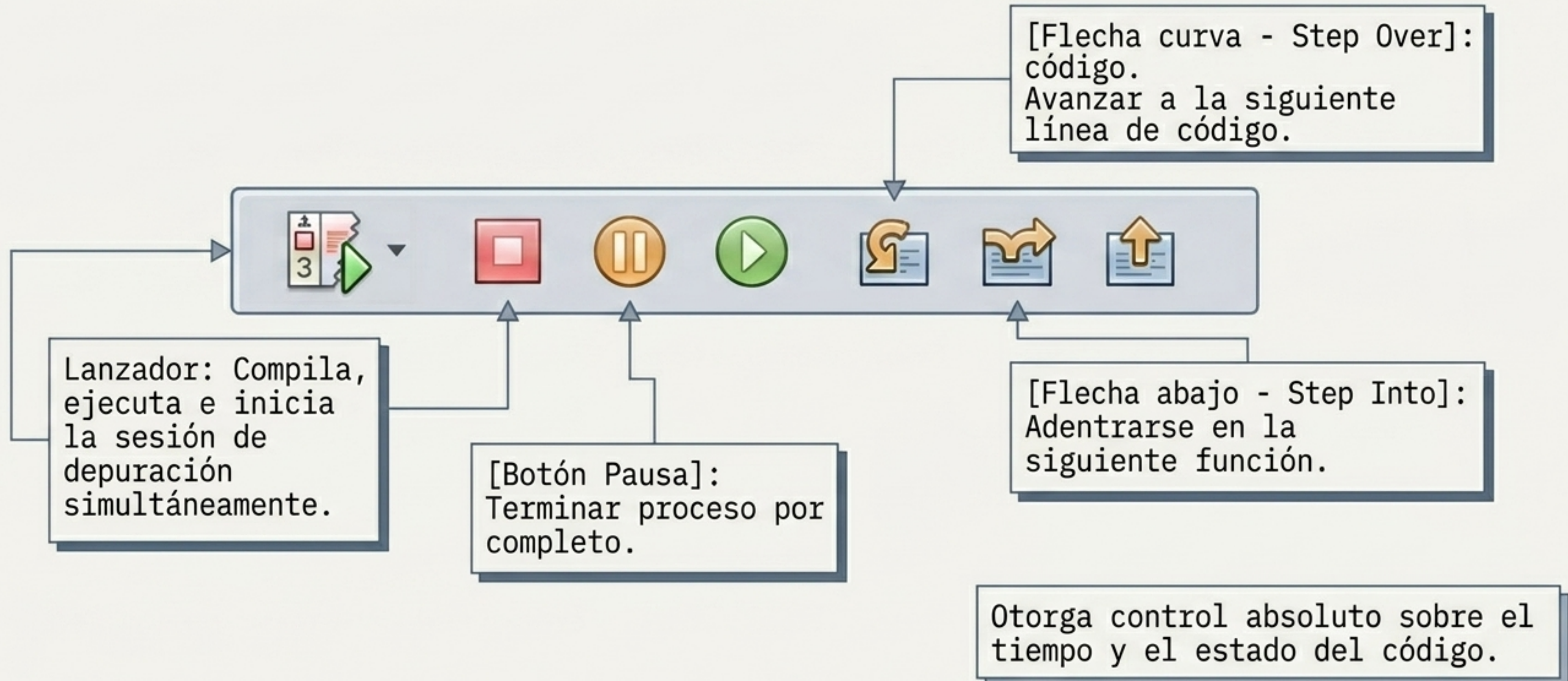


Asistencia inteligente. Uso de herramientas avanzadas para localizar anomalías estadísticamente.

El coste de la fiabilidad



Anatomía del control: Depurador de NetBeans



Congelar el tiempo: Puntos de Ruptura

El Breakpoint

40		try
41		{
42		System.out.println("Intro
43		edad = teclado_int.nextIn

El Breakpoint: Un stop intencional. La mejor estrategia es colocarlo justo antes de la línea sospechosa para interceptar el error en su origen.

Panel de Variables

Name	Type	Value
 Static		...
 args	String[]	... #288(length=0)
 teclado_int	Scanner	... #289
 edad	int	... 18

Panel de Variables: Inspección de constantes vitales. Al detenerse la ejecución, este panel revela el valor exacto de todas las variables implicadas en ese milisegundo preciso.

El cambio de paradigma: De reparar a prevenir

Las pruebas validan y verifican los requisitos del sistema de forma objetiva.

ENFOQUE DEL CREADOR



VISIÓN OBJETIVA

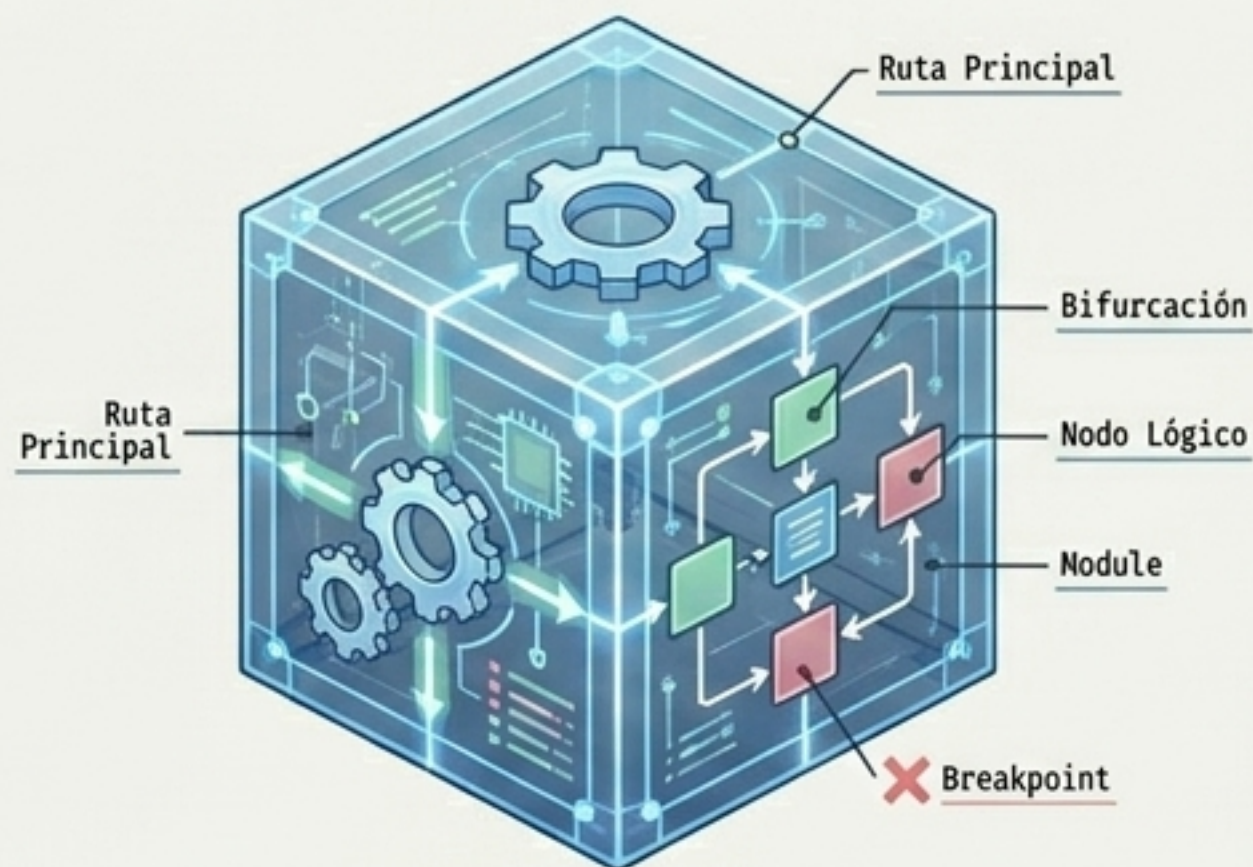


El sesgo del creador

Un programador casi nunca detecta los errores en el código que él mismo ha escrito. Las pruebas efectivas exigen una visión absolutamente independiente a la implementación.

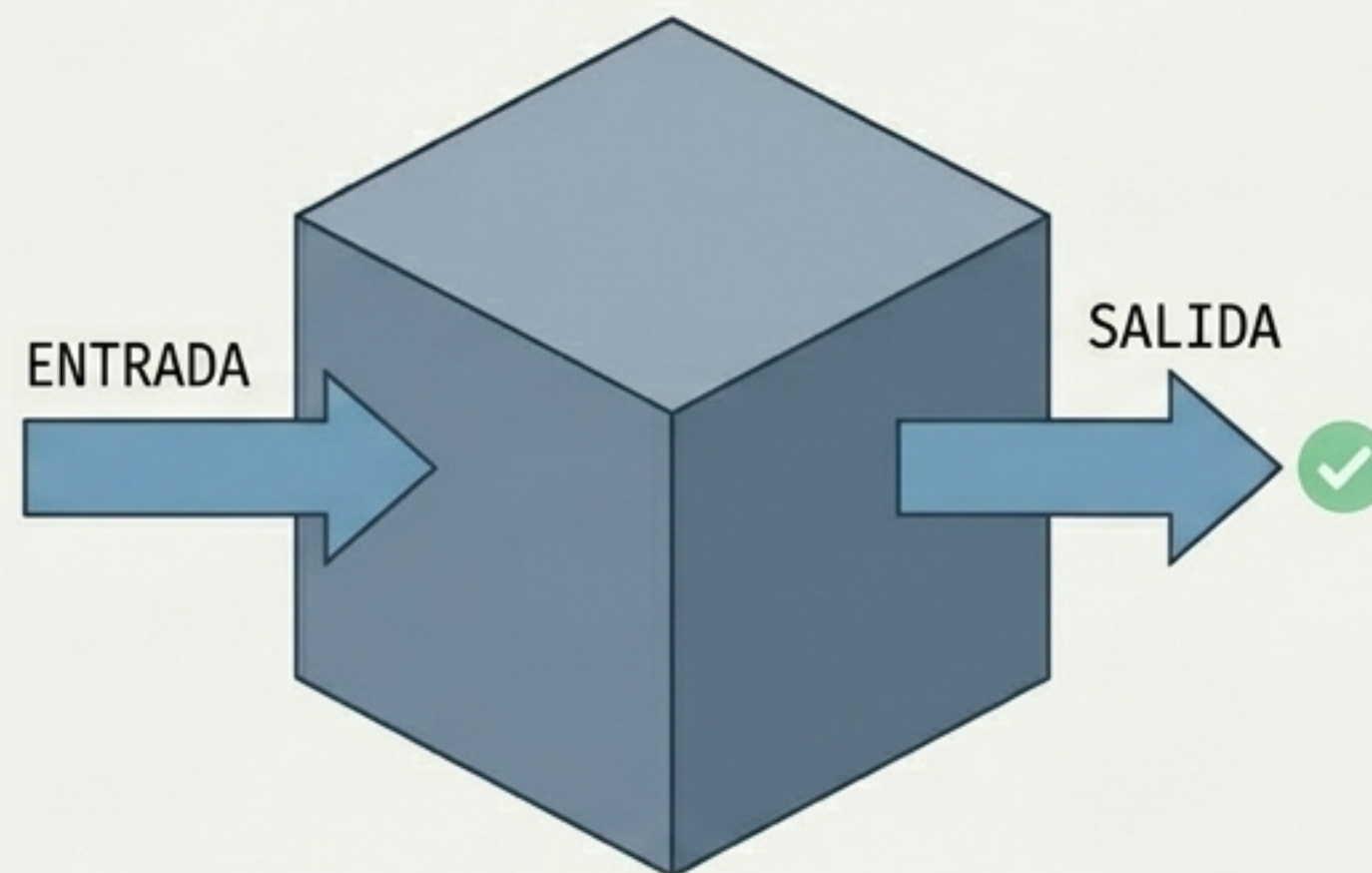
Anatomía de la Prueba: Caja Blanca vs. Caja Negra

Caja Blanca (Estructural)



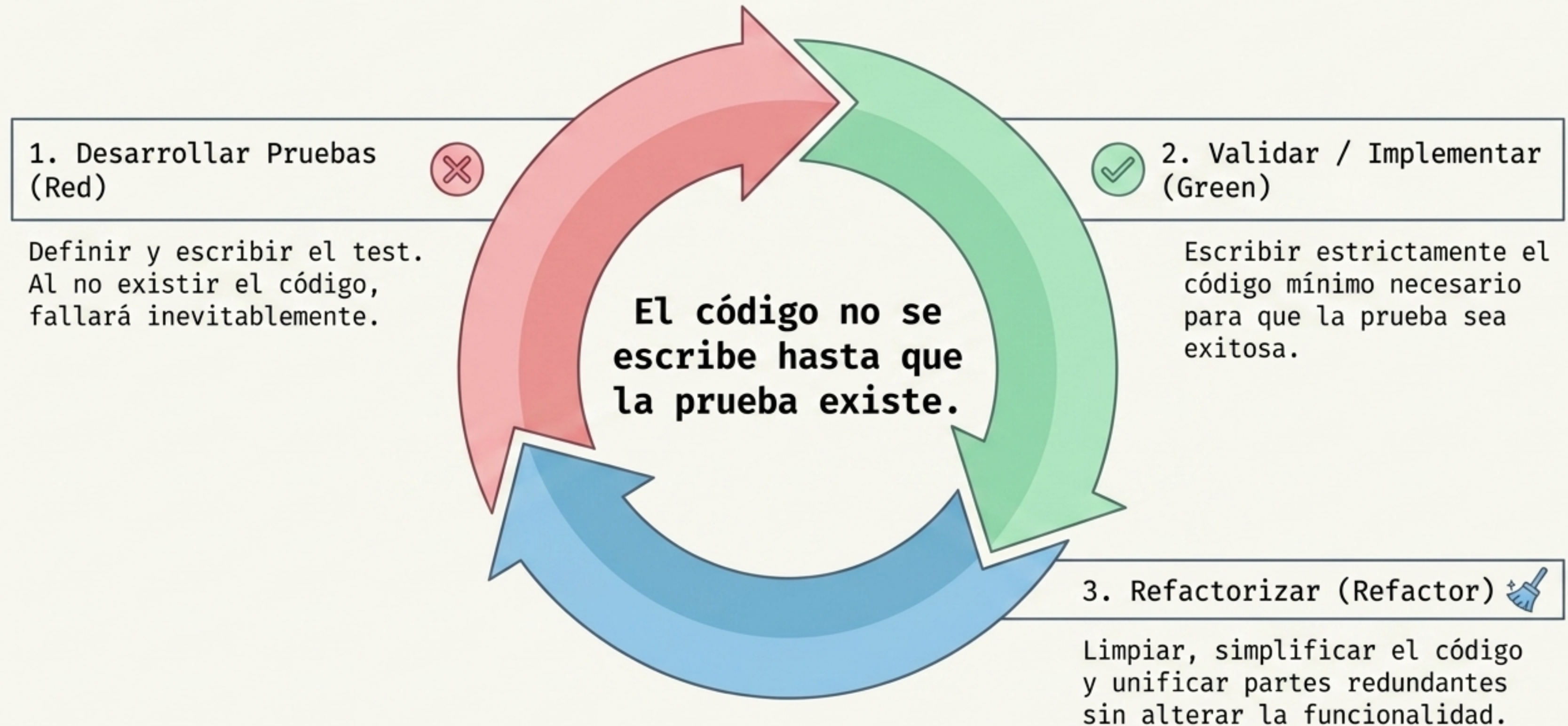
- Visibilidad: Total.
- Enfoque: Se conoce la arquitectura interna.
- Objetivo: Probar rutas, bifurcaciones, control de flujo y código fuente a nivel interno.

Caja Negra (Funcional)



- Visibilidad: Nula.
- Enfoque: Se ignora la implementación.
- Objetivo: Validar funcionalidades externas comprobando que un valor de entrada específico genera la salida esperada.

Invertir el proceso: Desarrollo Guiado por Pruebas (TDD)



Los tres pilares de la Calidad

Un software 100% libre de errores es imposible, pero la excelencia es gestionable.



Diagnósticos en el mundo real

TCK-01

Low Severity

Ticket 1: El Error Indetectable (Baja Gravedad)

Status: Open

Created: March 11, 2022

Problema: Caídas continuas en producción un mes después del despliegue. Imposible depurar el binario en vivo.

Solución experta: Jamás se depura producción directamente. Se insertan trazas remotas activables a demanda para registrar el flujo desde fuera.



TCK-02

High Severity

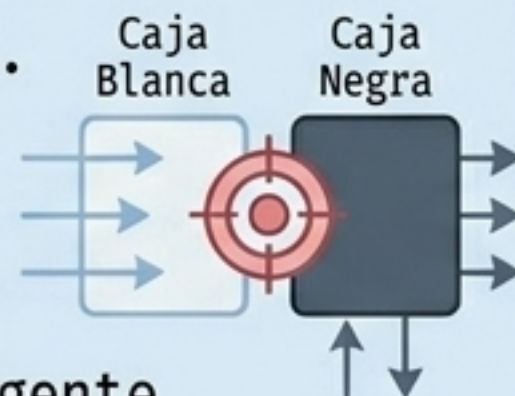
Ticket 2: Sistema de Vuelo Crítico (Peligro Vital)

Status: Open

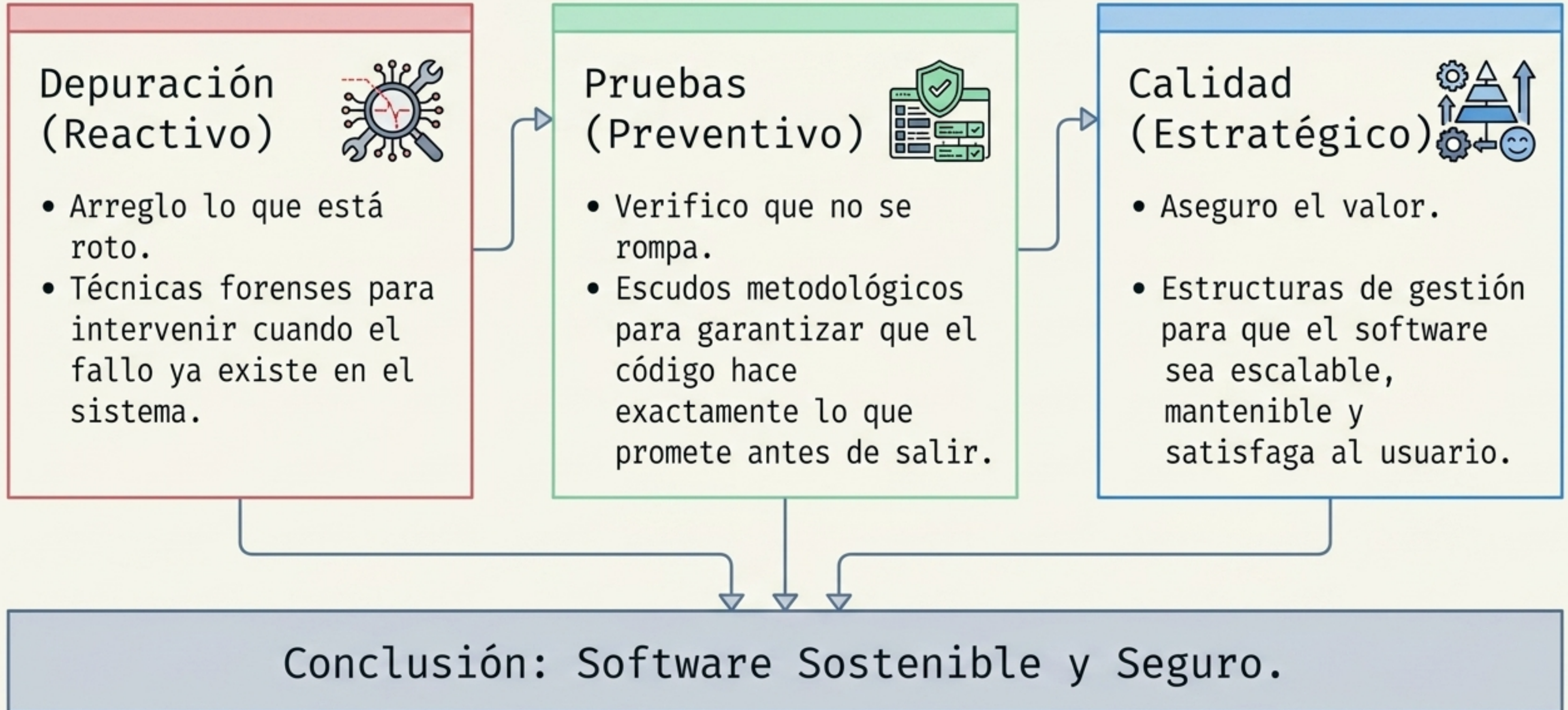
Created: March 11, 2022

Problema: Discordancia de sensores hace que un avión entre en pérdida.

Solución experta: Prioridad absoluta en la localización. Despliegue simultáneo de Caja Blanca y Negra con estrategias de eliminación. **Lección:** Localizar un fallo crítico es más difícil y urgente que escribir el parche.



El ecosistema de la fiabilidad



Arquitectura del Tema 15 (Resumen)

Depuración

Herramientas: Depurador (NetBeans), Breakpoints, Inspección.

Proceso: Identificación, Análisis, Corrección, Comprobación.

Estrategias: Fuerza bruta, Análisis Delante/Atrás, Eliminación.

Pruebas y Calidad

Técnicas de Pruebas: Unitarios, Integración, Monitorización.

Paradigmas: Caja Blanca (Estructural), Caja Negra (Funcional).

Metodología: TDD (Desarrollo Guiado por Pruebas).